1 整體系統架構

整個系統可以分成 **五層**：

```
Client Layer
     |
     ▼
Network Layer
     |
     ▼
Command Layer
     |
     ▼
Execution Layer
     |
     ▼
Storage Layer
```

同時有一個 **橫向模組**：

Trace Engine

完整圖：

```
             +----------------------+
             |        Web UI        |
             +----------▲-----------+
                        |
                Trace Query API
                        |
                        ▼


Client (redis-cli / custom cli)
            |
            ▼
     TCP Network Server
            |
            ▼
      Protocol Parser
```

```
        |
        ▼
Command Dispatcher
        |
        ▼
Command Executor
        |
        ▼
   Data Store
        |
        ▼
  TTL Manager

  ──────────────────────────
    Trace Engine
  ──────────────────────────
```

---

2 模組分層

❶ Network Layer

負責：

```
TCP connection
request read
response write
```

建議技術：

```
asyncio.start_server
```

模組：

```
network/
    server.py
    connection.py
```

server loop：

```
async def handle_client(reader, writer):
    data = await reader.read()
```

這層只負責 bytes。

不理解 command。

---

❷ Protocol Layer

解析 Redis protocol：

Redis 使用：

RESP (Redis Serialization Protocol)

例如：

```
*2
$3
GET
$3
key
```

Parser 輸出：

```
Command(
  name="GET",
  args=["key"]
)
```

模組：

```
protocol/
    resp_parser.py
    command.py
```

---

❸ Command Layer

負責：

command routing

例如：

```
GET -> get_handler
SET -> set_handler
```

設計：

```
command_table = {
   "GET": get_command,
   "SET": set_command,
}
```

模組：

```
commands/
    dispatcher.py
    string_commands.py
    key_commands.py
```

---

❹ Execution Layer

這裡是：

核心邏輯

例如：

```
SET key value
```

執行流程：

```
1 lookup key
2 allocate entry
```

```
3 write value
4 register ttl
```

模組：

```
engine/
    executor.py
```

Executor example：

```
async def execute(command):
    handler = command_table[command.name]
    return handler(command.args)
```

---

5 Storage Layer

Redis 本質：

```
in-memory key-value store
```

簡化設計：

```
dict
```

資料結構：

```
store = {
   key: Entry
}
```

Entry：

```
class Entry:
    value
    expire_at
```

模組：

```
storage/
    store.py
    entry.py
```

---

6 TTL Manager

負責：

過期 key

方法：

兩種選擇：

lazy expiration（推薦）

只有在：

GET key

時才檢查。

簡單很多。

未來可升級：

priority queue expiration

模組：

storage/ttl.py

---

7 Trace Engine（專案靈魂）

這是 **TraceRedis 的最大特色**。

所有 command 執行時：

都會生成 trace events。

概念：

```
command
 ├ parse
 ├ key lookup
 ├ write
 └ response
```

Trace Event 結構

```
class TraceEvent:
    timestamp
    event_type
    metadata
```

例如：

```
TraceEvent(
   event="hash_lookup",
   key="name"
)
```

Trace Session

每個 command：

都有一個 trace。

```
TraceSession
   command
   events[]
```

例如：

```
TraceSession
  command: SET name lin
  events:
    parse
    lookup
    write
```

模組：

```
trace/
    tracer.py
    session.py
    event.py
```

Trace API

trace engine 提供：

```
start_trace()
log_event()
end_trace()
```

example：

```
trace = tracer.start("SET")

tracer.event("hash_lookup", key="name")

tracer.event("write")

tracer.end()
```

---

8 Trace Storage

trace 要保存：

方便 UI 查看。

簡單方案：

memory ring buffer

例如：

last 1000 traces

模組：

trace/storage.py

---

9 Web Dashboard

前端顯示：

command list
trace timeline

API：

GET /traces
GET /trace/{id}

模組：

```
web/
    api.py
    dashboard/
```

前端：

React

顯示：

timeline
waterfall

---

10 CLI Client

提供一個：

```
traceredis-cli
```

例如：

```
SET name lin
TRACE
```

模組：

```
cli/
    client.py
```

---

3 完整目錄結構

建議：

```
traceredis/

network/
    server.py
    connection.py

protocol/
    resp_parser.py
    command.py

commands/
    dispatcher.py
    string.py
    key.py

engine/
    executor.py
```

```
storage/
    store.py
    entry.py
    ttl.py

trace/
    tracer.py
    session.py
    event.py
    storage.py

cli/
    client.py

web/
    api.py
    dashboard/

tests/
```

---

4 Request Flow（完整流程）

假設：

SET name lin

流程：

```
Client
  |
  ▼
TCP server
  |
  ▼
RESP parser
  |
```

```
  ▼
Command dispatcher
  │
  ▼
Executor
  │
  ├ trace.start
  │
  ├ storage lookup
  │
  ├ write value
  │
  ├ trace.event
  │
  └ trace.end
  │
  ▼
Response
```

---

5 Trace Visualization

trace example：

```
SET name lin

Trace:

parse           0.02ms
hash_lookup     0.03ms
write_value     0.01ms
register_ttl    0.01ms
response        0.01ms
```

Web UI：

```
| parse        |
| lookup       |
```

```
| write        |
| response     |
```

類似：

Chrome DevTools network waterfall。

---

6 MVP Scope（非常重要）

第一版 **不要做太多 Redis 功能**。

只做：

```
SET
GET
DEL
EXPIRE
KEYS
```

加上：

```
Trace engine
```

就已經很強。

---

7 最重要的設計原則

整個專案最重要的一句話：

Everything is traceable.

每個步驟都要：

```
emit trace event
```

這就是你的產品特色。

1 Trace Engine 的設計目標

TraceRedis 的 trace 系統要達成：

1 每個 command 都能產生 trace
2 每個 execution step 都能記錄 event
3 可以計算 latency
4 可以被 Web UI 查詢
5 overhead 不能太大

因此需要三個核心結構：

TraceSession
TraceEvent
TraceCollector

---

2 TraceSession（一次指令的完整 trace）

每個 command 都會產生一個 session。

例如：

SET name lin

會生成：

TraceSession

資料結構：

```
class TraceSession:
    id: int
    command: str
    start_time: float
    end_time: float
    events: list
```

範例：

```
TraceSession
 id: 42
 command: "SET name lin"
 start: 12.001
 end: 12.005

 events:
   parse
   lookup
   write
   response
```

這就是 UI 要顯示的資料。

---

3 TraceEvent（執行步驟）

每個 execution step 都是 event。

例如：

```
hash lookup
write value
```

資料結構：

```
class TraceEvent:
    name: str
    timestamp: float
    metadata: dict
```

example：

```
TraceEvent
 name="hash_lookup"
 timestamp=12.002
 metadata={
   "key": "name"
 }
```

另一個例子：

```
TraceEvent
 name="write_value"
 timestamp=12.003
 metadata={
   "size": 3
 }
```

---

4 為什麼要 metadata

metadata 讓 trace **更有分析價值**。

例如：

```
GET name
```

trace：

```
hash_lookup
key=name
bucket=17
```

或：

```
write_value
memory_bytes=32
```

UI 可以顯示更多資訊。

---

5 TraceCollector（管理所有 trace）

系統需要一個 central collector。

負責：

1 收集 trace
2 保存 trace
3 提供查詢

資料結構：

```
class TraceCollector:
    traces: list
    max_traces: int
```

為了避免 memory 爆掉：

建議使用：

```
ring buffer
```

例如：

```
max_traces = 1000
```

只保留最近 1000 個 command。

---

6 Ring Buffer 設計

原理：

新 trace 進來
如果滿了
刪掉最舊的

Python 可以用：

```
collections.deque(maxlen=1000)
```

這樣效能很好。

---

7 Tracer API（開發者使用的接口）

所有 command 都會用這些 API。

設計：

```
tracer.start(command)

tracer.event(name, **metadata)

tracer.end()
```

使用方式：

```
trace = tracer.start("SET name lin")

tracer.event("hash_lookup", key="name")

tracer.event("write_value")

tracer.end()
```

生成：

```
TraceSession
```

---

8 Context 設計（很重要）

server 是 async。

因此需要：

```
current trace context
```

Python 可以用：

```
contextvars
```

例如：

```
current_trace = ContextVar("trace")
```

start：

```
current_trace.set(session)
```

event：

```
session = current_trace.get()
```

這樣每個 request 都有自己的 trace。

---

9 latency 計算

不用每個 event 記 duration。

只記：

```
timestamp
```

UI 再計算：

```
delta = event[i+1] - event[i]
```

例如：

```
parse        0.02 ms
lookup       0.03 ms
write        0.01 ms
```

---

10 Trace 輸出格式（API）

Web UI 需要 JSON。

例如：

```
GET /trace/42
```

返回：

```
{
 "id": 42,
 "command": "SET name lin",
 "duration": 4,
 "events": [
   {"name": "parse", "t": 0},
   {"name": "lookup", "t": 1},
   {"name": "write", "t": 3}
 ]
}
```

UI 可以畫：

```
timeline
```

---

11 Trace 等級（進階設計）

可以設計 trace level：

```
OFF
BASIC
VERBOSE
```

例如：

```
BASIC
 parse
 execute
 response
VERBOSE
 hash lookup
 memory allocation
 ttl register
```

這讓專案看起來更 **專業**。

---

12 Trace Event 命名規則

建議固定命名：

```
parse_command
dispatch_command
hash_lookup
write_value
delete_key
ttl_register
response_send
```

這樣 UI 才好畫圖。

---

13 Trace 的完整流程

假設 command：

```
SET name lin
```

trace：

```
start trace

parse_command

dispatch_command

hash_lookup
 key=name

write_value
 size=3

response_send
```

end trace

UI：

```
parse       ■■■
lookup        ■■
write           ■
response          ■
```

---

14 效能問題

trace 可能影響效能。

解法：

可關閉 trace

例如：

```
TRACE OFF
TRACE ON
```

或 config：

```
enable_trace=True
```

---

15 Trace Engine 模組結構

建議：

```
trace/

  tracer.py
  session.py
  event.py
```

```
  collector.py
  context.py
```

職責：

```
tracer      API
session     trace data
event       event structure
collector   storage
context     request context
```

---

16 Example 完整 Python 結構

TraceEvent：

```
class TraceEvent:
    def __init__(self, name, timestamp, metadata=None):
        self.name = name
        self.timestamp = timestamp
        self.metadata = metadata or {}
```

TraceSession：

```
class TraceSession:
    def __init__(self, id, command):
        self.id = id
        self.command = command
        self.start_time = time.time()
        self.events = []
```

專案 Roadmap（TraceRedis）

整體開發流程：

```
v0.1   TCP server
v0.2   Redis protocol
v0.3   Key-value store
v0.4   Trace engine
v1.0   Visualization
```

時間大約：

6~8 weeks

Milestone 1

TCP Server Core

目標：

建立可以接受 client 連線的 server

任務：

Task 1

專案初始化

建立 repo：

```
traceredis/
```

資料夾：

```
traceredis/
network/
protocol/
commands/
```

```
storage/
trace/
cli/
web/
tests/
```

第一個 commit：

```
project skeleton
```

Task 2

TCP server

使用：

```
asyncio.start_server
```

server example：

```
async def handle_client(reader, writer):
    data = await reader.readline()
```

server 可以：

```
accept connection
print request
```

Task 3

connection manager

每個 client connection：

```
read loop
```

例如：

```
while True:
    read command
```

Task 4

basic response

server 回覆：

+OK\r\n

這是 RESP protocol。

完成 milestone 後：

v0.1 release

Milestone 2

Redis Protocol

目標：

server 可以理解 Redis command

Task 5

RESP parser

解析：

*2
$3
GET
$3
key

輸出：

```
Command(GET, ["key"])
```

Task 6

Command object

建立：

```
class Command:
    name
    args
```

Task 7

Command dispatcher

建立 routing：

```
command_table = {
  "GET": get_command,
  "SET": set_command
}
```

Task 8

Basic commands

先實作：

```
PING
ECHO
```

例如：

```
PING
→ PONG
```

完成 milestone：

v0.2 release

Milestone 3

In-Memory Database

目標：

完整 key-value store

Task 9

Storage engine

建立：

```
store = {}
```

Entry：

```
class Entry:
    value
    expire_at
```

Task 10

SET command

SET key value

流程：

lookup
store
return OK

Task 11

GET command

GET key

回：

value

Task 12

DEL command

DEL key

Task 13

TTL system

EXPIRE key seconds

簡單版本：

lazy expiration

完成 milestone：

v0.3 release

此時你已經有：

簡化版 Redis

Milestone 4

Trace Engine（專案核心）

目標：

每個 command 都可以被 trace

Task 14

TraceSession

實作：

```
class TraceSession
```

Task 15

TraceEvent

實作：

```
class TraceEvent
```

Task 16

TraceCollector

保存：

```
last 1000 traces
```

使用：

```
deque(maxlen=1000)
```

Task 17

Trace API

提供：

```
tracer.start()
tracer.event()
tracer.end()
```

Task 18

Command tracing

在 command execution 中加入：

```
parse_command
hash_lookup
write_value
response
```

完成 milestone：

v0.4 release

此時：

TraceRedis 已經有特色。

Milestone 5

Visualization

目標：

Trace 可以被可視化

Task 19

Trace REST API

提供：

```
GET /traces
GET /trace/{id}
```

返回 JSON。

Task 20

Web Dashboard

前端顯示：

```
command list
trace timeline
```

圖：

```
waterfall chart
```

類似：

Chrome DevTools。

完成 milestone：

```
v1.0 release
```

最終專案功能

TraceRedis：

Redis-like database
+ execution tracing
+ visualization

支持：

SET
GET
DEL
EXPIRE
KEYS

並且：

trace every command

GitHub Repo 最終樣子

README：

TraceRedis

A Redis-like database that visualizes how commands execute internally.

Demo：

SET name lin

Trace:

parse_command
hash_lookup
write_value
response_send

Web UI：

timeline

一個很重要的 GitHub 技巧

每完成 milestone：

建立：

GitHub release

例如：

v0.1 TCP server
v0.2 RESP parser
v0.3 KV store
v0.4 Trace engine
v1.0 Dashboard

這會讓 repo **非常專業**。